bomblab 实验报告

陈睿尹

2026 年 10 月 24 日

1 实验环境

本实验在 Ubuntu 24.04.5 LTS x86-64 环境中完成，使用 GCC 编译，使用 GDB 和 objdump 进行调试与反汇编分析。BombLab 仓库中的 `bomb` 已正常领取，编译和测试均通过。整个实验只修改了 `defuser.c`，正式六关与 Honor 均已完成。

2 分析过程

我先按实验指导使用 objdump 得到各关卡函数及其调用函数的反汇编，再通过 GDB 在函数入口和关键比较指令处查看寄存器、栈帧和内存。分析时主要关注参数寄存器、比较指令前的计算结果、循环中的递推关系、跳转表以及函数指针表。对于 32 位运算，统一按照无符号整数回绕处理。

3 正式关卡分析

3.1 Phase 1

这一关只输入一个无符号整数。入口函数先调用 `valid_u32_fields` 检查输入格式，然后把 `p1_compute` 的返回值和输入比较：

```
call   p1_compute
cmp    %eax,-0x14(%rbp)
sete   %al
```

继续查看 `p1_compute` 可以发现它依次调用了 `p1_step1`、`p1_step0` 和 `p1_step2`。结合其中的 `sub`、`lea`、`xor` 等指令，可以恢复出下面的关系。记 challenge 的低 32 位、高 32 位和右移 16 位后的值分别为 l、h、m，则：

$$t_1 = 3(h - m) + \texttt{0x9864},$$
$$t_0 = (l + 2h) \oplus \texttt{0x209c},$$
$$ans = (3t_0 + t_1) \oplus \texttt{0xb95b}.$$

这里的所有运算都是 32 位无符号运算。实现时直接把这三个步骤写入 `solve_phase_1`，输出一个整数。

3.2 Phase 2

从 `sscanf` 附近的代码可以看出本关需要 6 个整数。第一个数的计算比较直接：

```
lea    0x0(%r13,%r13,1),%eax
and    $0x7e,%eax
add    $0x9,%eax
cmp    %eax,%edx
```

后面的循环中可以看到移位、掩码、异或和加法指令：

```
shl    $0x2,%edx
shr    %cl,%rax
and    $0x1f,%eax
xor    %edx,%eax
add    %edi,%eax
cmp    %eax,%edx
```

整理后，六个整数满足：

$$x_0 = ((2 \cdot challenge)\ \&\ \texttt{0x7e}) + 9,$$
$$x_i = (((challenge \gg (i+2))\ \&\ \texttt{0x1f}) \oplus (x_{i-1} \ll 2)) + 10i, \quad i = 1, \ldots, 5.$$

循环中当前的 `edx` 正好是上一个结果的低 32 位，所以按这个递推式依次计算即可。

3.3 Phase 3

本关需要输入两个整数。第一个输入必须等于 `challenge & 7`，随后程序计算一个参数并调用 `p3_dispatch`：

```
and    $0x7,%edi
cmp    %edi,-0x18(%rbp)
...
shr    $0x3,%rax
shr    $0x20,%rsi
xor    %eax,%esi
call   p3_dispatch
```

记：

$$idx = challenge\ \&\ 7, \qquad arg = (challenge \gg 3) \oplus (challenge \gg 32).$$

`p3_dispatch` 通过跳转表选择分支。需要特别注意的是，跳转表中的分支并不是按 case 编号顺序排列的。根据实际跳转目标整理后，第二个输入应满足：

idx	计算关系
0	$5 \cdot arg + \texttt{0x345c1}$
1	$(arg \oplus \texttt{0x130f}) + (arg \gg 2)$
2	$3 \cdot arg + \texttt{0x8a95}$
3	$5 \cdot arg + \texttt{0xd9ee}$
4	$(arg \oplus \texttt{0x9387}) + (arg \gg 2)$
5	$(4 \cdot arg) \oplus \texttt{0xde42}$
6	$(2 \cdot arg) \oplus \texttt{0xf75e}$
7	$5 \cdot arg + \texttt{0x12ca}$

3.4 Phase 4

本关输入一个整数，程序通过递归函数 `p4_walk` 计算期望值。该函数每次取当前区间的中点，如果中点不等于目标值，就根据目标值在左半边还是右半边继续递归：

```
sub    %esi,%edi
shr    $1,%edi
add    %esi,%edi
cmp    %eax,%edi
je     402330
```

递归返回时，左右分支会分别异或不同的常量：

```
lea    (%rax,%rax,2),%eax
xor    %ebx,%eax
xor    $0x14,%eax
```

恢复出的递归关系如下：

```
mid = lo + ((hi - lo) >> 1);

if (mid == target)
    return (mid + 0x9b) ^ key;

mask = mid + key;

if (target < mid)
    return (3 * walk(target, lo, mid - 1, key)) ^ mask ^ 0x19;
else
    return (3 * walk(target, mid + 1, hi, key)) ^ mask ^ 0x14;
```

初始值为：

$$target = challenge \,\&\, \texttt{0x7f}, \qquad key = (challenge \gg 7) \oplus (challenge \gg 32),$$

并且 $lo = 0$，$hi = \texttt{0x7f}$。

实现时我没有另外写递归辅助函数，而是用数组保存每一层的异或常量，到达目标后再从最深层向浅层回代，结果与递归写法一致。

3.5 Phase 5

本关输入两个整数。程序根据 `challenge & 7` 从 `0x405b20` 处的指针数组中选择起始节点，然后沿着链表向后遍历：

```
mov    0x405b20(,%rax,8),%rdx
mov    0x8(%rdx),%esi
xor    0xc(%rdx),%eax
add    %esi,%eax
mov    (%rdx),%rdx
```

通过 objdump 查看该地址附近的数据后可以确认，每个节点占 16 字节：

偏移	含义
+0	next 指针
+8	第一个 32 位整数
+12	第二个 32 位整数

恢复出的计算过程为：

```
node = table[challenge & 7];

acc = (challenge >> 6)
      ^ (challenge >> 32)
      ^ node->field12
      ^ 0x6ece;

acc += node->field8;
second_answer = node->field8;

steps = (challenge >> 3) & 3;

for (i = 1; i <= steps; i++) {
    node = node->next;
    acc ^= node->field12;
    acc += node->field8;
    second_answer = node->field8;
}

first_answer = acc;
```

最终两个输入分别等于 `first_answer` 和 `second_answer`。节点数据是在只读段中读取的，不需要运行炸弹就能恢复。

3.6 Phase 6

本关输入一个整数。入口函数先计算一个混合值，再通过函数指针表调用两次：

```
call   *0x405dc0(,%rax,8)
xor    $0xe282,%eax
mov    %eax,%edi
call   *0x405dc0(%rbx)
```

```
add    $0x1e3e,%eax
```

根据函数指针表可以整理出 8 个索引对应的运算：

索引	函数功能
0, 1	$(x \oplus \texttt{0xff6b}) + 8x$
2	$((x \gg 2) \oplus x) + \texttt{0x740f}$
3, 5	$\mathrm{rol}(x, 5) \oplus \texttt{0x8db3}$
4	$\mathrm{ror}(x, 3)$
6	$(x + \texttt{0xc395}) \oplus (x \gg 7)$
7	$(8x + \texttt{0xf50f}) \oplus 3x$

两次调用的索引分别来自 `challenge` 的低 3 位和第 3 到第 5 位。设：

$$x = (challenge \gg 6) \oplus (challenge \gg 32),$$

则：

$$first = op[challenge \,\&\, 7](x) \oplus \texttt{0xe282},$$
$$ans = op[(challenge \gg 3) \,\&\, 7](first) + \texttt{0x1e3e}.$$

4 Honor 分析

Honor 入口先检查整行长度，并检查偏移 `0x20` 处的四个字节是否为 `"OPEN"`：

```
sub    $0x24,%rax
cmp    $0x7f,%rax
...
cmpl   $0x4e45504f,-0xb0(%rbp)
jne    ...
```

因此第七行需要满足：

前 32 字节填充内容 + "OPEN" + 3 个无符号十进制整数

偏移 `0x24` 之后的三个整数由 `secret_phase` 检查。设 `context` 是由前六关答案生成的值，则：

```
c = context & 3;
chmix = (challenge >> 32) ^ challenge;
ctxmix = (context >> 32) ^ context;
mixarg = chmix ^ ctxmix;

mixed = ((mixarg >> 1) ^ 0xd907)
        + (mixarg ^ 0x30cd);

v1 = c;
v2 = mixed ^ (c * 0x30cd ^ (context >> 16));
v3 = v2 ^ (ctxmix + c + 0xd907 + (chmix >> 1));
```

最后输出一行：

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPENv1 v2 v3
```

前 32 个字符可以任选非空字符，只要保证整行长度合法、偏移 0x20 处为 OPEN，并且后面正好是三个整数即可。

5 测试与验证

修改完 defuser.c 后重新执行 make，编译没有报错。运行公开测试，结果为：

```
Public tests: 60/60
Secret public tests: 10/10 (optional)
```

公开测试通过截图如下：



图 1: 公开测试 60/60 并通过 Honor

除公开测试外，我还用 seed 43 做了完整验证。正式六关均显示 defused，Honor 显示 Secret phase defused.。具体记录如下：

关卡	验证 seed	结果
Phase 1	43	通过
Phase 2	43	通过
Phase 3	43	通过
Phase 4	43	通过
Phase 5	43	通过
Phase 6	43	通过
Honor	43	通过

seed 43 的运行截图如下：

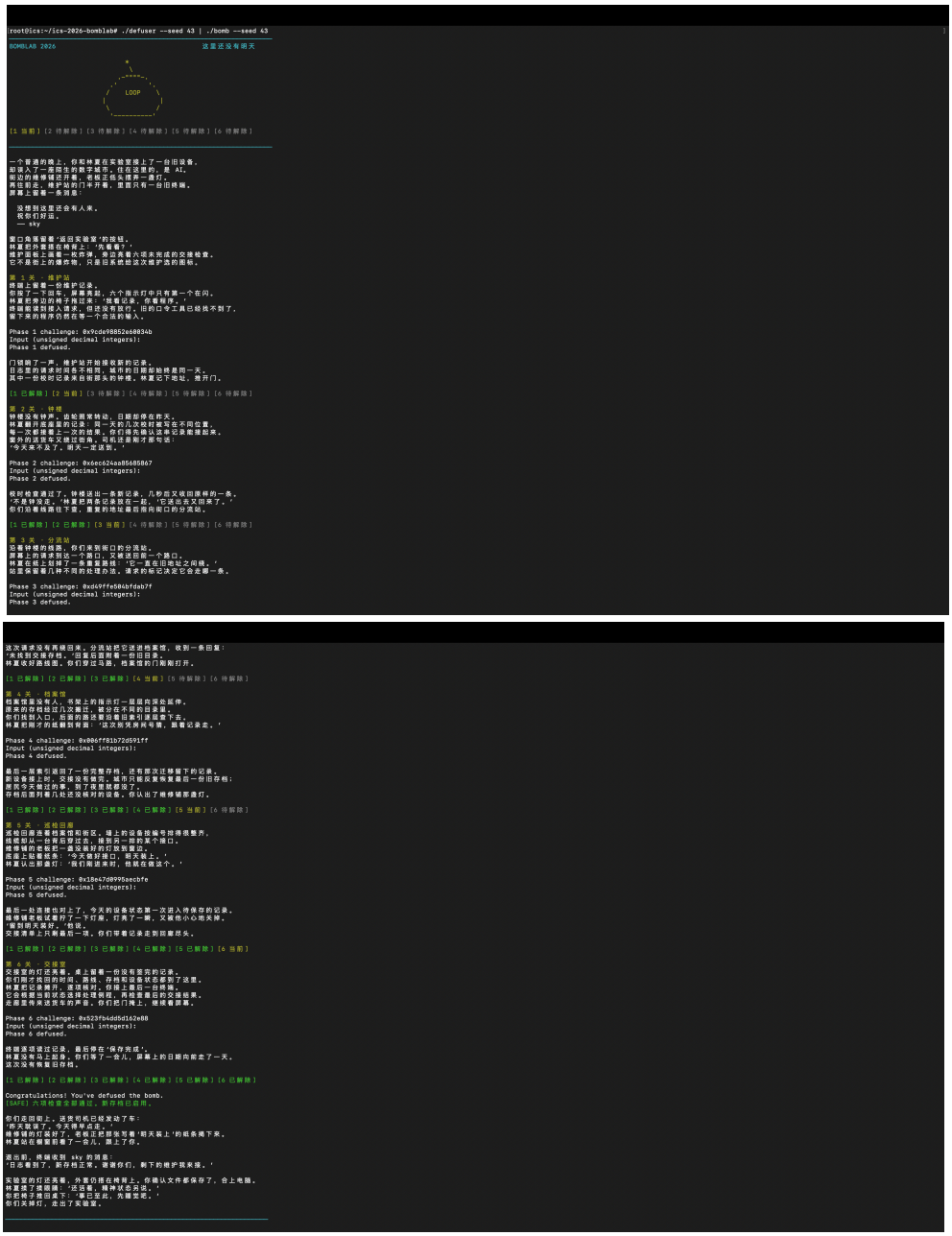


图 2: seed 43 正式六关全部通过

这说明解法不是只对某一个 seed 有效，而是能够根据 challenge 重新计算口令。

6 参考资料

1. 《深入理解计算机系统》，第 3 章：程序的机器级表示。

2. 课程网站 BombLab 部分。

3. GNU GDB Documentation。

4. GNU objdump / binutils Documentation。

7 辅助工具使用说明

在个别较难理解的汇编语句以及报告文字润色过程中使用了 AI 工具辅助；最终代码和运行结果均在本地环境中编译、运行并复核。